PROJE TASARIM RAPORU

Necmettin Erbakan Üniversitesi

Mühendislik Fakültesi — Bilgisayar Mühendisliği Bölümü

BİL 204 — Yazılım Mühendisliği

Restoran Yönetim Sistemi (RYS)

Ders: BİL 204 — Yazılım Mühendisliği

Bölüm/Şube: Bilgisayar Mühendisliği

Proje Kısa Adı: RYS

Teslim Tarihi: 04.05.2026

Grup Üyeleri:

Ramazan Faruk Acıöz — 24100011046

İssa Abakar Mahamat — 23100011701

Mehmet Selim Ceran — 24100011054

Hüseyin Kaya — 24100011066

Muhammed Yusuf Uğur — 24100011043

Serdar Kara — 24100011081

Ders Sorumlusu: Öğr. Gör. Arda Söylev

Nisan 2026

İçindekiler ..... Error! Bookmark not defined.
1. Giriş 3
1.1. Sistemin Amacı 3
1.2. Tasarım Hedefleri 3
1.2.1. Performans ve Gerçek Zamanlı İletişim 3
1.2.2. Kullanılabilirlik (Usability) 3
1.2.3. Güvenlik (Security) 3
1.2.4. Güvenilirlik ve Bakım Yapılabilirlik 3
2. Üst Düzey Yazılım Mimarisi 4
2.1. Alt Sistem Ayrıştırması 4
2.2. Donanım/Yazılım Eşlemesi 4
2.3. Kalıcı Veri Yönetimi 5
2.4. Erişim Kontrolü ve Güvenlik 6
2.5. Küresel Kontrol Akışı 7
2.6. Sınır Koşulları 7
2.6.1. Başlatma 8
2.6.2. Sonlandırma 8
2.6.3. Hata Durumları 8
3. Alt Düzey Tasarım 9
3.1. Nesne Tasarım Kararları 9
Karar 1: Veritabanı Erişimi — ORM mi, Ham SQL (pg) mi? 9
Karar 2: Frontend Durum Yönetimi — Redux mu, useState mi? 9
Karar 3: Gerçek Zamanlı KDS Güncellemeleri — HTTP Polling mi, WebSocket mi? 9
3.2. Son Nesne Tasarımı 9
3.3. Paketler / Katmanlar 10
3.3.1. Kullanıcı Arayüzü Yönetim Katmanı 10
3.3.2. Web Sunucu Katmanı 11
3.3.3. Veri Yönetim Katmanı 12
3.4. Sınıf Arayüzleri 13
3.4.1. AuthController 13
3.4.2. OrdersController (routes/orders.js) 14
3.4.3. ProductsController (routes/products.js) 14
3.4.4. TablesController (routes/tables.js) 15
3.4.5. UsersController (routes/users.js) 15
3.4.6. db/pool.js (Singleton pg.Pool) 16
3.5. Tasarım Desenleri 16
Sözlük (Glossary) 18
Referanslar 20

1. Giriş

Bu tasarım raporu, BİL 204 — Yazılım Mühendisliği dersi kapsamında geliştirilen Restoran Yönetim Sistemi'nin (RYS) mimari ve bileşen düzeyindeki tasarımını belgelemektedir. Analiz raporu 'sistem ne yapacak?' sorusunu yanıtlarken bu rapor 'sistem bunu nasıl yapacak?' sorusunu yanıtlamaktadır.

1.1. Sistemin Amacı

RYS, orta ölçekli bir restoranın günlük operasyonlarını — sipariş alma ve KDS'e iletmeden ödeme ve fatura kesimine, otomatik stok takibinden yönetimsel raporlamaya kadar — uçtan uca dijitalleştiren web tabanlı çok katmanlı bir yazılım çözümüdür. Konya'da faaliyet gösteren bir restoranda dört paydaş grubuyla (garson, mutfak şefi, kasiyer, restoran sahibi) gerçekleştirilen yüz yüze görüşmeler sonucunda tespit edilen başlıca sorunlar şunlardır: kağıt fişlere dayalı sipariş akışı, mutfak-salon iletişim kopukluğu, sezgiye dayalı stok yönetimi ve manuel ödeme-fatura süreçleri. Proje kaynak kodu https://github.com/rmznfaruk/rys-bil204 adresinde yönetilmektedir.

1.2. Tasarım Hedefleri

SRS belgesinde tanımlanan işlevsel olmayan gereksinimlerden dört temel tasarım hedefi türetilmiştir.

1.2.1. Performans ve Gerçek Zamanlı İletişim

Sipariş onayından KDS'de görünmesine kadar geçen süre 2 saniyenin altında olmalıdır. Sistem eş zamanlı en az 50 aktif kullanıcıyı desteklemelidir. Raporlar 5 saniye içinde oluşturulabilmeli; tüm kullanıcı girdileri 2 saniye içinde onaylanmalıdır. Bu hedef orders.js'deki ACID transaction yapısı ve pg bağlantı havuzu ile desteklenmektedir. Gerçek zamanlı KDS güncellemeleri 2. iterasyonda Socket.io WebSocket ile sağlanacaktır.

1.2.2. Kullanılabilirlik (Usability)

Arayüz 30 dakikalık eğitimle kullanılabilir olmalı; sipariş girişi en fazla 3 adımda tamamlanabilmelidir. Dokunmatik ekrana uygun büyük butonlar kullanılmalıdır. Masa durumları renk kodlamasıyla anlık gösterilmelidir: boş (yeşil), dolu (kırmızı), rezerveli (sarı), temizleniyor (gri). Bu hedef 7 sayfa bileşeni ve React Router 7 ile hayata geçirilmektedir.

1.2.3. Güvenlik (Security)

Her kullanıcı rolü yalnızca kendi yetki alanındaki modüllere erişebilmelidir. 5 ardışık hatalı girişte hesap 15 dakika kilitlenmelidir; şifreler BCrypt ile hash'lenmeli; ödeme verileri AES-256 ile saklanmalıdır. 1. iterasyonda hatalı giriş kilitleme personel.hatali_giris ve personel.kilit_bitis alanlarıyla veritabanı seviyesinde uygulanmaktadır. JWT kimlik doğrulaması ve BCrypt şifre hash'leme 2. iterasyona planlanmıştır.

1.2.4. Güvenilirlik ve Bakım Yapılabilirlik

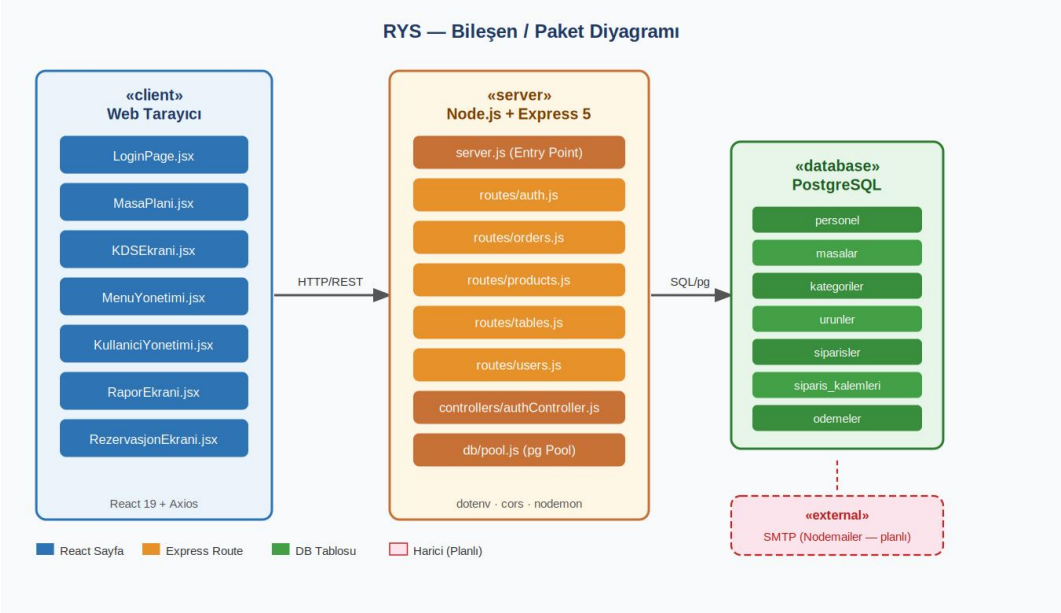
Sistem haftada en fazla 10 dakika hizmet dışı kalabilir (≥ %99,9 erişilebilirlik). Ödeme işlemleri sırasında veri kaybı yaşanmamalıdır; orders.js'deki BEGIN/COMMIT/ROLLBACK transaction zinciri bunu garanti altına almaktadır. Veritabanı yedeklemesi 12 saatte bir otomatik gerçekleştirilmelidir. Menüye yeni ürün eklenmesi yazılım güncellemesi gerektirmemelidir; sistem REST API ile üçüncü parti entegrasyonlara açık olmalıdır.

2. Üst Düzey Yazılım Mimarisi

Bu bölümde sistemin genel mimarisi, alt sistem ayrıştırması, donanım/yazılım eşlemesi, kalıcı veri yönetimi, erişim kontrolü, kontrol akışı ve sınır koşulları ele alınmaktadır.

2.1. Alt Sistem Ayrıştırması

RYS, endişelerin ayrılması (separation of concerns) ilkesine dayalı beş katmanlı bir mimari ile tasarlanmıştır. Kullanıcı Arayüzü Katmanı (React 19 SPA), Kimlik Doğrulama Katmanı (authController.js), Web Sunucu Katmanı (Node.js + Express 5), Veri Erişim Katmanı (pg Pool — db/pool.js) ve Veri Tabanı Katmanı (PostgreSQL 15+) bu beş katmanı oluşturmaktadır. Aşağıdaki diyagram alt sistemleri ve aralarındaki ilişkileri göstermektedir.

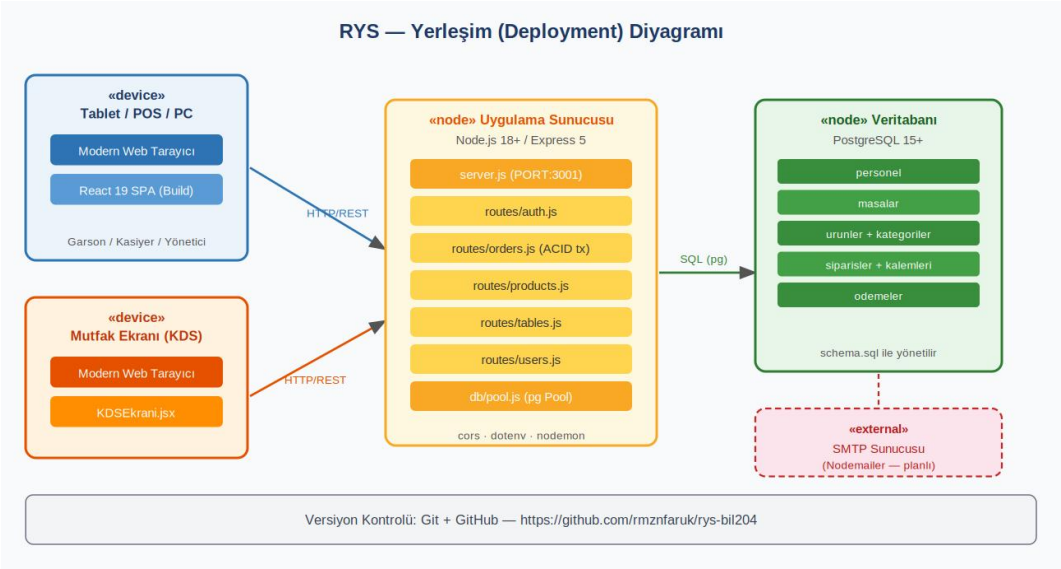


Şekil 1: RYS Bileşen / Paket Diyagramı

Katman	Teknoloji	Sorumluluk
Kullanıcı Arayüzü	React 19 (SPA)	7 sayfa bileşeni; Axios HTTP iletişimi; rol bazlı sayfa yönetimi; React Router 7
Kimlik Doğrulama	authController.js + routes/auth.js	Giriş/çıkış; hatali_giris ve kilit_bitis kontrolü; 2. iterasyonda JWT
Web Sunucu	Node.js 18+ / Express 5.2	5 route modülü (auth, orders, products, tables, users); REST API; cors, dotenv
Veri Erişim	pg 8.20 Pool — db/pool.js	PostgreSQL bağlantı havuzu; ham SQL; ACID transaction (BEGIN/COMMIT/ROLLBACK)
Veri Tabanı	PostgreSQL 15+	7 tablo; schema.sql DDL; bire-çok ilişkiler; ON DELETE CASCADE

2.2. Donanım/Yazılım Eşlemesi

Sistem web tabanlı olduğundan istemci tarafında modern bir tarayıcı yeterlidir. Tablet/POS terminali garson ve kasiyer için, masaüstü yönetici için, büyük duvar ekranı mutfak için kullanılır. Aşağıdaki yerleşim diyagramı fiziksel bileşenlerin dağılımını göstermektedir.

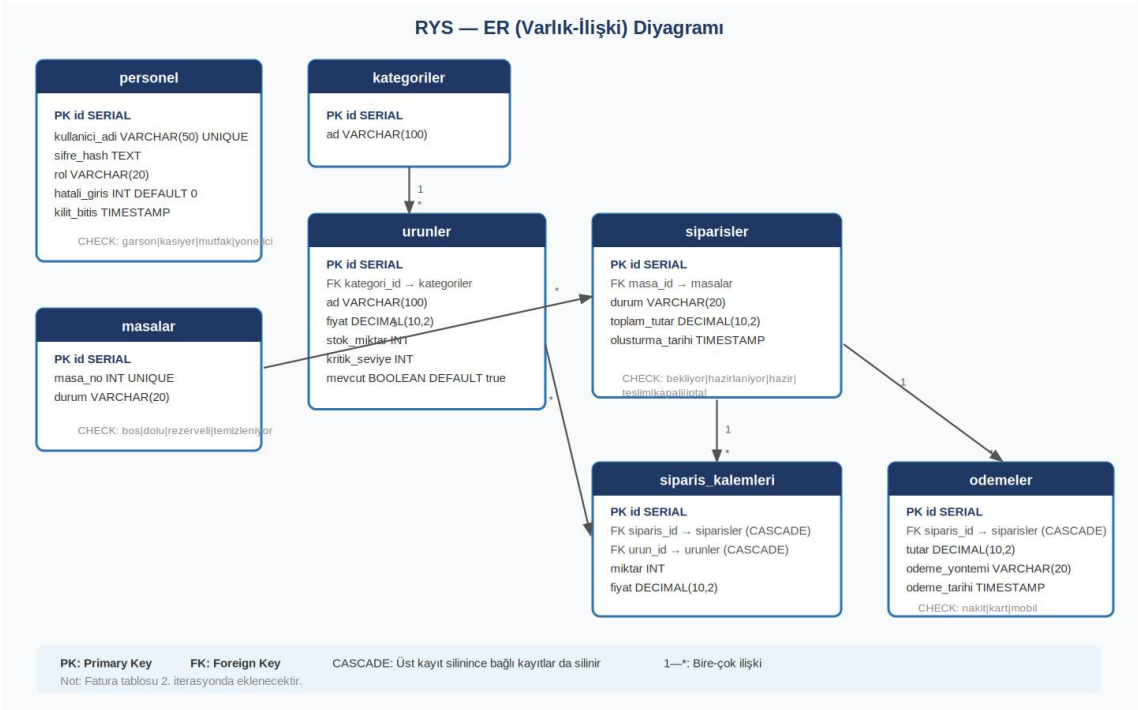


Şekil 2: RYS Yerleşim (Deployment) Diyagramı

Donanım Bileşeni	Yazılım / Teknoloji	Kullanıcı	Açıklama
Tablet / POS Terminali	React 19 SPA — Tarayıcı	Garson, Kasiyer	MasaPlani, sipariş girişi, ödeme; dokunmatik ekran uyumlu
Masaüstü PC / Laptop	React 19 SPA — Tarayıcı	Yönetici	Tüm yönetim modülleri; raporlama ve kullanıcı yönetimi
Büyük Duvar Ekranı	React 19 — KDSEkrani.jsx	Mutfak Personeli	Aktif sipariş kartları; bekleme sayacı; 'Hazır' işaretleme
Uygulama Sunucusu	Node.js 18+ / Express 5	—	REST API; server.js; PORT 3001; min. 2 CPU, 2 GB RAM
Veritabanı Sunucusu	PostgreSQL 15+	—	ACID; 7 tablo; schema.sql; 12 saatlik otomatik yedekleme
SMTP Sunucusu (Planlı)	Nodemailer (2. iterasyon)	—	Stok kritik seviye e-posta uyarıları; yöneticiye bildirim

2.3. Kalıcı Veri Yönetimi

ACID uyumluluğu ve güçlü ilişkisel yapısı nedeniyle PostgreSQL 15+ seçilmiştir. Tüm tablolar db/schema.sql dosyasında DDL olarak tanımlanmış; pg (node-postgres) kütüphanesi üzerinden ham SQL sorguları ile yönetilmektedir. ORM tercih edilmemiştir; bu karar Bölüm 3.1'de gerekçelendirilmiştir.



Şekil 3: RYS Varlık-İlişki (ER) Diyagramı

Veri yönetiminin temel tasarım kararları şunlardır:

- Sipariş oluşturma atomic transaction içinde yapılır: BEGIN → INSERT siparisler → INSERT siparis_kalemleri (döngü) → UPDATE masalar SET durum='dolu' → COMMIT. Herhangi bir adımda hata olursa ROLLBACK tüm değişiklikleri geri alır; finally bloğunda client.release() ile bağlantı havuza iade edilir.
- Fiyat sipariş kalemine anlık kopyalanır (snapshot); menü fiyat değişiklikleri geçmiş siparişleri etkilemez.
- Stok yönetimi urunler.stok_miktar (anlık miktar) ve urunler.kritik_seviye (uyarı eşiği) ile sağlanır. Stok düşme işlemi 2. iterasyonda sipariş transaction'ına dahil edilecektir.
- Tüm yabancı anahtar ilişkilerinde ON DELETE CASCADE kullanılır; üst kayıt silinince bağlı tüm alt kayıtlar otomatik silinir.
- Fatura tablosu 2. iterasyona planlanmıştır; şu an odemeler tablosu ödeme kaydını tutar.

2.4. Erişim Kontrolü ve Güvenlik

Dört kullanıcı rolü tanımlanmıştır: garson, mutfak, kasiyer, yönetici. Roller personel.rol sütununda saklanmakta ve kimlik doğrulama akışında authController.js tarafından kontrol edilmektedir. Aşağıdaki erişim kontrol matrisi her rolün hangi işleve ve API uç noktasına erişebildiğini göstermektedir.

İşlev / API Uç Noktası	Garson	Mutfak	Kasiyer	Yönetici
POST /api/auth/login — Giriş Yap	✓	✓	✓	✓
POST /api/orders — Sipariş Al	✓			
PATCH /api/orders/:id — Sipariş Güncelle/İptal	✓			
PATCH /api/tables/:id — Masa Durumu Güncelle	✓			✓

GET /api/orders — KDS Sipariş Görüntüle		✓		✓
PATCH /api/orders/:id (hazir) — Hazır İşaretle		✓		
POST /api/payments — Hesabı Kapat (planlı)	✓		✓	
GET+POST+PUT+DELETE /api/products — Menü Yönet				✓
GET+POST+PUT+DELETE /api/users — Kullanıcı Yönet				✓
GET /api/reports — Raporlar (planlı)				✓
Stok Uyarısı E-postası — SMTP (planlı)				✓

Güvenlik uygulama detayları: Hatalı giriş kilitleme, authController.js içinde personel.hatali_giris sayacı 5'e ulaştığında personel.kilit_bitis alanını 15 dakika ileriye set ederek uygulanmaktadır. Her giriş denemesinde önce kilit_bitis kontrolü yapılır; kilit süresi dolmamışsa giriş reddedilir. JWT tabanlı kimlik doğrulama, BCrypt şifre hash'leme ve AES-256 ödeme şifrelemesi 2. iterasyona planlanmıştır. Şu an sifre_hash sütunu kullanılmakta; hash işlemi 2. iterasyonda eklenecektir.

2.5. Küresel Kontrol Akışı

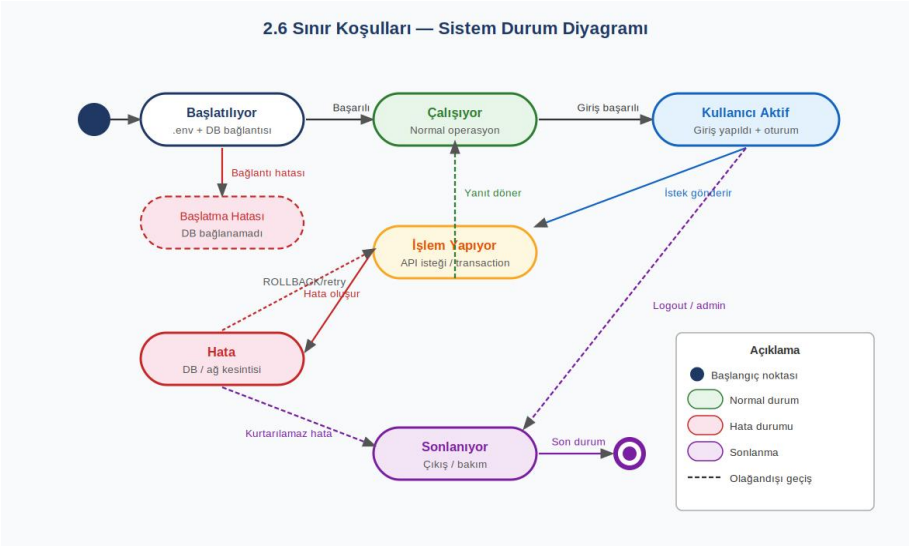
RYS, olay güdümlü (event-driven) kontrol modeli benimser. Sistem iki ana iletişim kanalı ile çalışmaktadır:

- HTTP / REST (Senkron — 1. iterasyon): Kullanıcı eylemleri Axios HTTP istekleriyle Express router'a iletilir. İstek ilgili route modülüne (auth.js, orders.js, products.js, tables.js, users.js) yönlendirilerek db/pool.js üzerinden SQL çalıştırılır ve JSON yanıt döner. Kritik işlemler pool.connect() → BEGIN/COMMIT/ROLLBACK zinciriyle ACID transaction garantisiyle korunur.
- WebSocket / Socket.io (Asenkron — 2. iterasyon): Gerçek zamanlı KDS güncellemeleri ve garson 'sipariş hazır' bildirimleri Socket.io ile sağlanacaktır. Şu an KDSEkrani.jsx periyodik GET /api/orders çağrısıyla aktif siparişleri çekmektedir.

Hata yönetimi: Tüm beklenmedik hatalar Express hata işleme middleware'i tarafından yakalanır; yığın izleri (stack trace) istemciye açık edilmez, yalnızca sunucu loglarına yazılır. Ödeme transaction'ı sırasında hata oluşursa ROLLBACK otomatik çalışır.

2.6. Sınır Koşulları

Aşağıdaki durum diyagramı sistemin yaşam döngüsündeki tüm sınır koşullarını göstermektedir.



Şekil 4: RYS Sınır Koşulları Durum Diyagramı

2.6.1. Başlatma

Sistem başlatılırken dotenv, .env dosyasındaki ortam değişkenlerini yükler (DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, PORT). db/pool.js singleton pg.Pool nesnesini oluşturur ve PostgreSQL bağlantısını test eder; bağlantı kurulamazsa sunucu hata mesajıyla durur. İlk çalıştırmada db/schema.sql veritabanına uygulanarak 7 tablo ve kısıtlamalar oluşturulur. React SPA, src/frontend/build/ klasöründeki statik dosyalardan sunulur; kullanıcı LoginPage sayfasıyla karşılanır.

2.6.2. Sonlandırma

Kullanıcı POST /api/auth/logout çağrısında oturum bilgisi temizlenir; 2. iterasyonda JWT token kara listeye eklenecektir. SRS gereksinimi olarak 30 dakika işlem yapılmadığında oturum otomatik kapatılması 2. iterasyonda implement edilecektir. Aktif bir sipariş transaction'ı devam ederken sunucu kapatılırsa PostgreSQL transaction'ı otomatik ROLLBACK yapar; veri tutarsızlığı oluşmaz. Nodemon geliştirme modunda dosya değişikliklerinde sunucuyu yeniden başlatır.

2.6.3. Hata Durumları

Veritabanı bağlantısı koptuğunda tüm API uç noktaları 500 Internal Server Error döner; istemci ekranında kullanıcı dostu mesaj gösterilir. Sipariş oluşturma sırasında hata oluşursa (unique constraint, ağ kesintisi, stok yetersizliği) orders.js ROLLBACK çalıştırır ve client.release() ile bağlantıyı havuza iade eder; hem sipariş hem masa durumu tutarlı kalır. SMTP sunucusu erişilemez olduğunda stok uyarısı kuyruğa alınarak yeniden denenir (2. iterasyon). ON DELETE CASCADE kısıtlaması sayesinde üst kayıt silindiğinde bağlı alt kayıtlar otomatik temizlenir; orphan kayıt oluşmaz.

3. Alt Düzey Tasarım

Bu bölümde nesne tasarım kararları, son nesne tasarımı UML sınıf diyagramıyla, katman diyagramları, sınıf arayüzleri ve kullanılan tasarım desenleri ayrıntılı olarak açıklanmaktadır.

3.1. Nesne Tasarım Kararları

Tasarım sürecinde üç temel ödünleşim kararı verilmiştir. Her karar alternatiflerin karşılaştırılmasıyla belgelenmiştir.

Karar 1: Veritabanı Erişimi — ORM mi, Ham SQL (pg) mi?

Değerlendirilen iki alternatif şunlardır:

- Alternatif A — Sequelize ORM: Model soyutlaması ve migration yönetimi sağlar; ancak ek bağımlılık, öğrenme eğrisi ve performans ek yükü getirir.
- Alternatif B — pg (node-postgres) ile Ham SQL: Doğrudan PostgreSQL özelliklerine erişim; tam kontrol; ek katman yok.

Karar: pg seçilmiştir. Gerekçe: orders.js'deki sipariş oluşturma akışı — pool.connect() → BEGIN → INSERT siparisler → INSERT siparis_kalemleri (döngü) → UPDATE masalar SET durum='dolu' → COMMIT / ROLLBACK → client.release() — bu seviyede granüler ACID transaction kontrolü gerektirir; ORM bu karmaşıklığı şeffaf biçimde yönetemez. Ödünleşim: şema yönetimi schema.sql'de elle yapılır, model katmanı yoktur. Kazanım: her sorgunun tam kontrolü ve güvenilir transaction yönetimi.

Karar 2: Frontend Durum Yönetimi — Redux mu, useState mi?

Değerlendirilen iki alternatif şunlardır:

- Alternatif A — Redux: Merkezi store, time-travel debugging, güçlü araç desteği; ancak projemizin ölçeği için aşırı karmaşıklık.
- Alternatif B — React useState (bileşen bazlı durum): Bağımlılık yok, sade yapı, hızlı geliştirme.

Karar: useState seçilmiştir. Gerekçe: mevcut 7 sayfa bileşeninin bağımsız veri ihtiyaçları bileşen bazlı durum ile karşılanmaktadır. Bileşenler arası paylaşılan durum gerekirse React Context API eklenecektir. Ödünleşim: Redux'un gelişmiş debug araçlarından vazgeçilmiştir. Kazanım: sade kod tabanı, hızlı geliştirme, sıfır ekstra bağımlılık.

Karar 3: Gerçek Zamanlı KDS Güncellemeleri — HTTP Polling mi, WebSocket mi?

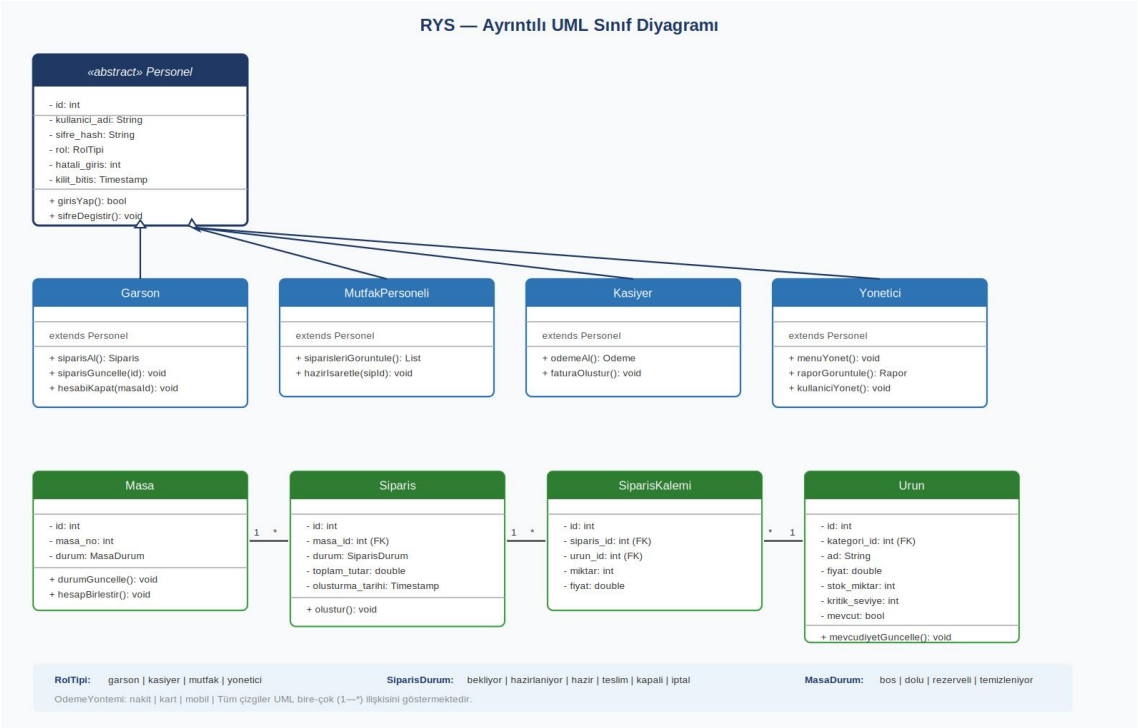
Değerlendirilen iki alternatif şunlardır:

- Alternatif A — HTTP Polling: GET /api/orders'ı her N saniyede bir çağırır; basit kurulum, ancak gereksiz ağ trafiği ve gecikme.
- Alternatif B — Socket.io WebSocket: Kalıcı bağlantı ile anlık yayın; daha düşük gecikme ve sunucu yükü; ancak ek kütüphane ve bağlantı yönetimi gerektirir.

Karar: 1. iterasyonda polling, 2. iterasyonda Socket.io. Gerekçe: 1. iterasyonun önceliği temel işlevselliği çalışır kılmaktır; WebSocket karmaşıklığı 2. iterasyona ertelenmiştir. SRS'deki '2 saniyede KDS iletimi' gereksinimi Socket.io ile karşılanacaktır. Ödünleşim: geçici gecikme ve gereksiz istek. Kazanım: erken teslimat ve sade başlangıç mimarisi.

3.2. Son Nesne Tasarımı

Aşağıdaki UML sınıf diyagramı sistemin tüm sınıflarını, özniteliklerini, metotlarını ve ilişkilerini göstermektedir. Analiz raporundaki UML sınıf diyagramına kıyasla gerçek PostgreSQL schema.sql'den türetilen alan adları ve veri tipleri kullanılmıştır; ilişki çoklukları ve CASCADE kuralları eklenmiştir.



Şekil 5: RYS Ayrıntılı UML Sınıf Diyagramı

Personel soyut temel sınıftır; Garson, MutfakPersoneli, Kasiyer ve Yonetici alt sınıfları kalıtım alır. Her alt sınıf kendi rol davranışlarını uygular. Masa ile Siparis arasında bire-çok ilişki vardır; sipariş oluşturulduğunda masa durumu 'dolu' yapılır. Siparis ile SiparisKalemi arasında bire-çok ilişki; her kalem bir Urun'a referans verir ve fiyatın anlık kopyasını saklar. Urun, Kategori ile çoka-bir ilişkidedir. Siparis ile Odeme arasında bire-çok ilişki vardır.

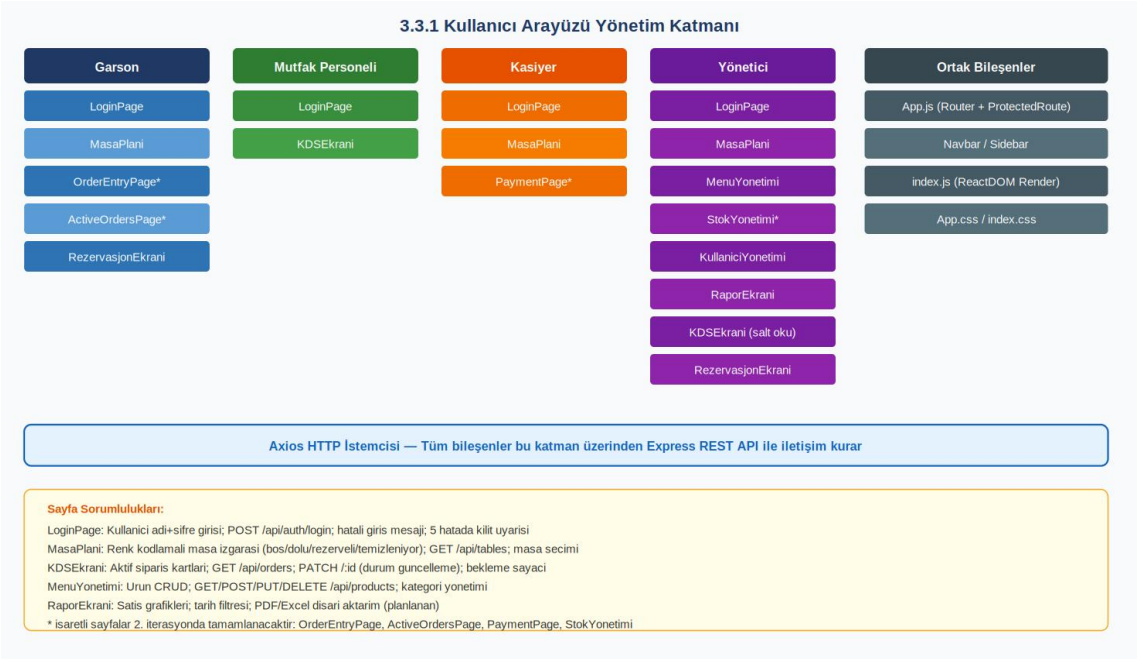
Enum değerleri: SiparisDurum — bekliyor, hazirlaniyor, hazir, teslim, kapali, iptal. MasaDurum — bos, dolu, rezerveli, temizleniyor. OdemeYontemi — nakit, kart, mobil. RolTipi — garson, mutfak, kasiyer, yonetici.

3.3. Paketler / Katmanlar

Sistem üç ana katmana ayrılmaktadır: Kullanıcı Arayüzü Yönetim Katmanı (React SPA), Web Sunucu Katmanı (Node.js/Express), Veri Yönetim Katmanı (PostgreSQL + pg). Her katman için ayrıntılı bileşen diyagramı aşağıda verilmektedir.

3.3.1. Kullanıcı Arayüzü Yönetim Katmanı

Bu katman, React 19 ile geliştirilmiş tüm sayfa bileşenlerini kapsar. Her sayfa Axios HTTP istemcisi üzerinden Express REST API ile iletişim kurar. Rol bazlı erişim kontrolü App.js'deki ProtectedRoute bileşeniyle sağlanır.

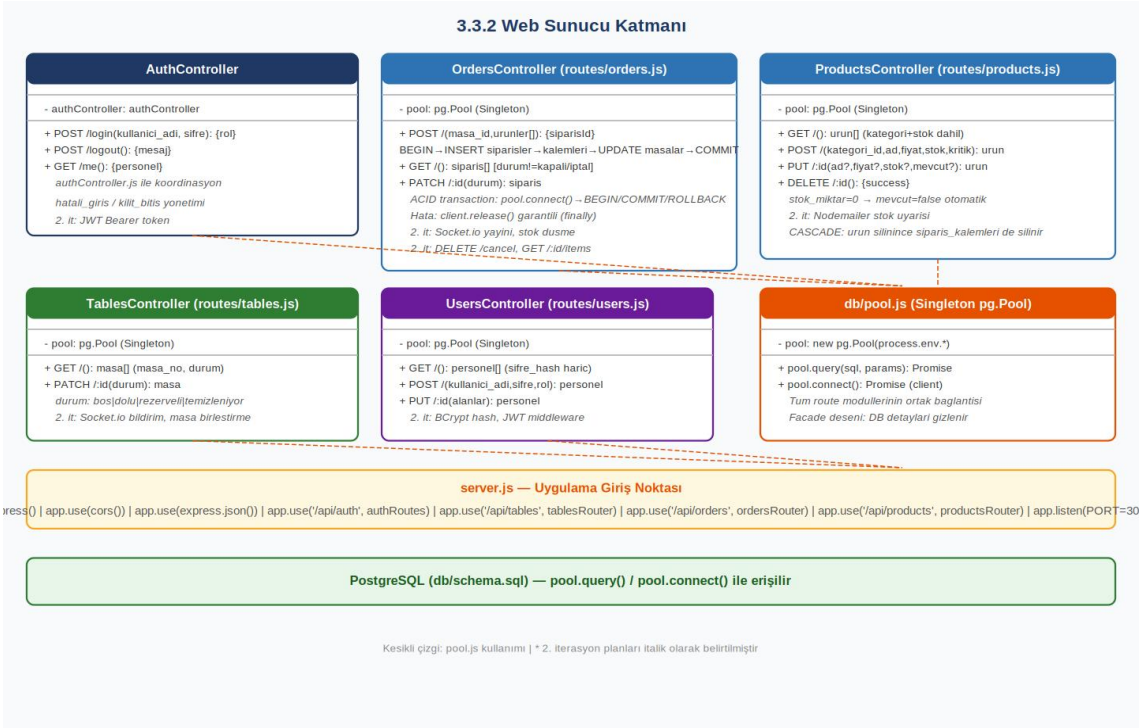


Şekil 6: Kullanıcı Arayüzü Katmanı Diyagramı

Bileşen (Dosya)	Erişen Rol(ler)	Kullandığı API Uç Noktaları	Sorumluluk
LoginPage.jsx	Tümü	POST /api/auth/login	Kullanıcı adı/şifre; hata mesajı; kilit uyarısı; rol bazlı yönlendirme
MasaPlani.jsx	Garson, Kasiyer, Yönetici	GET /api/tables, PATCH /api/tables/:id	Renk kodlamalı masa ızgarası; durum güncelleme; masa seçimi
KDSEkrani.jsx	Mutfak, Yönetici	GET /api/orders, PATCH /api/orders/:id	Aktif sipariş kartları; bekleme sayacı; 'Hazır' işaretleme
MenuYonetimi.jsx	Yönetici	GET/POST/PUT/DELETE /api/products	Ürün ve kategori CRUD; fiyat ve stok güncelleme
KullaniciYonetimi.jsx	Yönetici	GET/POST/PUT/DELETE /api/users	Personel hesapları yönetimi; rol atama
RaporEkrani.jsx	Yönetici	GET /api/reports (planlı)	Satış grafikleri; tarih filtresi; PDF/Excel dışa aktarım
RezervasjonEkrani.jsx	Garson, Yönetici	PATCH /api/tables/:id	Masa rezervasyon yönetimi; rezervasyon oluşturma/iptal
App.js	—	—	React Router 7 rota tanımları; ProtectedRoute; genel layout

3.3.2. Web Sunucu Katmanı

Bu katman Node.js + Express 5 ile geliştirilmiş tüm route ve controller modüllerini kapsar. Katmanın tek veri erişim noktası db/pool.js'teki pg.Pool singleton nesnesidir; tüm SQL sorguları bu havuz üzerinden çalıştırılır.

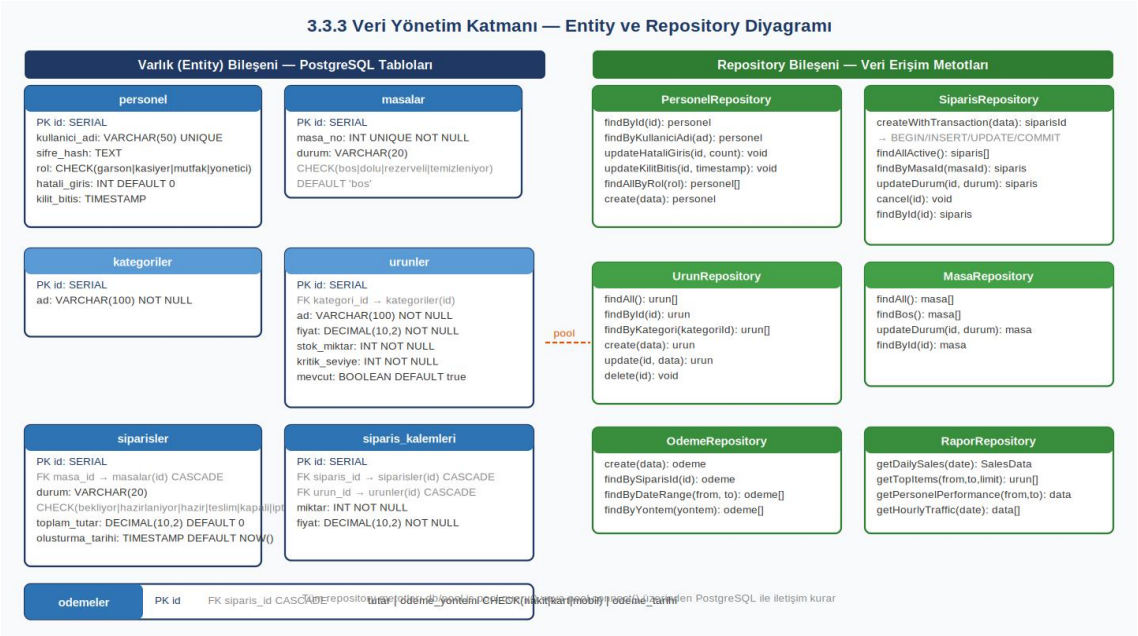


Şekil 7: Web Sunucu Katmanı Diyagramı

Modül (Dosya)	HTTP Yöntemleri / Uç Nokta	Temel İşlev
authController.js + routes/auth.js	POST /api/auth/login, POST /api/auth/logout	Kullanıcı doğrulama; hatali_giris/kilit_bitis yönetimi; rol döndürme
routes/orders.js	POST /, GET /, PATCH /:id	ACID transaction ile sipariş oluşturma; aktif sipariş listesi; durum güncelleme
routes/products.js	GET /, POST /, PUT /:id, DELETE /:id	Menü ürünleri CRUD; stok_miktar ve mevcut yönetimi; CASCADE delete
routes/tables.js	GET /, PATCH /:id	Masa listesi ve durum güncelleme; bos|dolu|rezerveli|temizleniyor
routes/users.js	GET /, POST /, PUT /:id, DELETE /:id	Personel CRUD; sifre_hash (2. it. BCrypt); yalnızca yönetici
db/pool.js	pool.query() / pool.connect()	Singleton pg.Pool; .env bağlantı config; tüm route'ların ortak noktası
server.js	—	Express kurulumu; cors, express.json(); tüm route'lar mount edilir; PORT 3001

3.3.3. Veri Yönetim Katmanı

Bu katman PostgreSQL 15+ veritabanını ve veri erişim metotlarını kapsar. Her varlık (entity) için bir Repository sınıfı tanımlanmıştır. Tüm repository metotları db/pool.js üzerinden çalışır.



Şekil 8: Veri Yönetim Katmanı — Entity ve Repository Diyagramı

Repository	Temel Metotlar	İlgili Tablo(lar)	Açıklama
PersonelRepository	findById, findByKullaniciAdi, updateHataliGiris, updateKilitBitis, findAllByRol, create	personel	Kimlik doğrulama ve kilitlenme mantığı; kullanıcı CRUD
SiparisRepository	createWithTransaction, findAllActive, findByMasaId, updateDurum, cancel, findById	siparisler, siparis_kalemleri	ACID transaction zinciri; KDS için aktif sipariş sorgusu
UrunRepository	findAll, findById, findByKategori, create, update, delete	urunler, kategoriler	Menü CRUD; stok_miktar ve mevcut yönetimi
MasaRepository	findAll, findBos, updateDurum, findById	masalar	Masa durumu takibi; MasaPlani.jsx için veri kaynağı
OdemeRepository	create, findBySiparisId, findByDateRange, findByYontem	odemeler	Ödeme kaydı ve raporlama sorguları
RaporRepository	getDailySales, getTopItems, getPersonelPerformance, getHourlyTraffic	siparisler, siparis_kalemleri, odemeler	Yönetici raporlama; tarih bazlı agregasyon sorguları

3.4. Sınıf Arayüzleri

Bu bölümde her katmandaki önemli sınıfların metot imzaları, parametreleri, dönüş tipleri ve kısa açıklamaları verilmektedir. Örnek raporla uyumlu olarak öznitelikler (attributes) ve metotlar birlikte sunulmaktadır.

3.4.1. AuthController

Öznitelik / Metot	Tip / İmza	Açıklama
— Öznitelikler —		
pool	pg.Pool (Singleton)	db/pool.js'ten alınan pg bağlantı havuzu
— Metotlar —		
login(req, res)	POST /api/auth/login → {rol, mesaj} | 401 | 423	kullanici_adi ile sorgu; kilit_bitis kontrolü; hatali_giris artırma; başarıda rol döndürme
logout(req, res)	POST /api/auth/logout → {mesaj}	Oturum temizleme; 2. iterasyonda JWT token kara liste
getMe(req, res)	GET /api/auth/me → personel	Oturum bilgisi döndürme; 2. iterasyonda JWT decode

3.4.2. OrdersController (routes/orders.js)

Öznitelik / Metot	Tip / İmza	Açıklama
— Öznitelikler —		
pool	pg.Pool (Singleton)	db/pool.js'ten alınan pg bağlantı havuzu
— Metotlar —		
createOrder(req, res)	POST / body:{masa_id, urunler:[{id,miktar,fiyat}]} → 201:{siparisId}	pool.connect() → BEGIN → INSERT siparisler → INSERT siparis_kalemleri (for loop) → UPDATE masalar SET durum='dolu' → COMMIT; hata → ROLLBACK; finally: client.release()
getActiveOrders(req, res)	GET / → 200:siparis[]	SELECT * FROM siparisler WHERE durum NOT IN ('kapali','iptal') ORDER BY olusturma_tarihi ASC; KDS için
updateOrderStatus(req, res)	PATCH /:id body:{durum} → 200:siparis	UPDATE siparisler SET durum=$1 WHERE id=$2 RETURNING *; durum geçiş kuralları uygulanır

3.4.3. ProductsController (routes/products.js)

Öznitelik / Metot	Tip / İmza	Açıklama
— Öznitelikler —		
pool	pg.Pool (Singleton)	db/pool.js'ten alınan pg bağlantı havuzu

— Metotlar —		
getAllProducts(req, res)	GET / → 200:urun[]	SELECT u.*, k.ad AS kategori FROM urunler u JOIN kategoriler k ON u.kategori_id=k.id; stok_miktar ve mevcut dahil
createProduct(req, res)	POST / body:{kategori_id,ad,fiyat,stok_miktar,kritik_seviye} → 201:urun	INSERT INTO urunler; mevcut varsayılan true
updateProduct(req, res)	PUT /:id body:{ad?,fiyat?,stok_miktar?,mevcut?} → 200:urun	UPDATE urunler; stok_miktar=0 ise mevcut otomatik false
deleteProduct(req, res)	DELETE /:id → 200:{success}	DELETE FROM urunler CASCADE; bağlı siparis_kalemleri de silinir

3.4.4. TablesController (routes/tables.js)

Öznitelik / Metot	Tip / İmza	Açıklama
— Öznitelikler —		
pool	pg.Pool (Singleton)	db/pool.js'ten alınan pg bağlantı havuzu
— Metotlar —		
getAllTables(req, res)	GET / → 200:masa[]	SELECT * FROM masalar ORDER BY masa_no ASC; MasaPlani.jsx için
updateTableStatus(req, res)	PATCH /:id body:{durum} → 200:masa	UPDATE masalar SET durum=$1 WHERE id=$2 RETURNING *; CHECK kısıtı geçersiz durumu reddeder

3.4.5. UsersController (routes/users.js)

Öznitelik / Metot	Tip / İmza	Açıklama
— Öznitelikler —		
pool	pg.Pool (Singleton)	db/pool.js'ten alınan pg bağlantı havuzu
— Metotlar —		
getAllUsers(req, res)	GET / → 200:personel[]	SELECT id,kullanici_adi,rol FROM personel; sifre_hash döndürülmez; yalnızca yönetici

createUser(req, res)	POST / body:{kullanici_adi,sifre,rol} → 201:personel	INSERT INTO personel; 2. iterasyonda bcrypt.hash(sifre,10) uygulanacak
updateUser(req, res)	PUT /:id body:{...} → 200:personel	UPDATE personel SET ... WHERE id=$n RETURNING *; kısmi güncelleme destekli
deleteUser(req, res)	DELETE /:id → 200:{success}	DELETE FROM personel; 2. iterasyonda soft delete (aktif=false) uygulanacak

3.4.6. db/pool.js (Singleton pg.Pool)

Öznitelik / Metot	Tip / İmza	Açıklama
— Öznitelikler —		
pool	new pg.Pool({host,port,database,user,password})	process.env.* değerleriyle oluşturulan tek pg.Pool örneği
— Metotlar —		
pool.query(sql, params)	Promise<QueryResult>	Basit sorgular için; havuzdan otomatik client alır ve iade eder
pool.connect()	Promise<PoolClient>	Transaction gerektiren işlemler için; client.query() ile BEGIN/COMMIT/ROLLBACK; finally: client.release()
module.exports = pool	pg.Pool	Singleton export; tüm route modülleri require('../db/pool') ile bu örneği kullanır

3.5. Tasarım Desenleri

Projede beş tasarım deseni uygulanmış ya da planlanmıştır. Her desen için hangi sınıf(lar)da ve nasıl uygulandığı açıklanmaktadır.

Desen	Uygulanan Sınıf / Dosya	Uygulama Yöntemi	Gerekçe ve Kazanım
Singleton	db/pool.js	module.exports = new pg.Pool(...) — tek örnek ihraç edilir; tüm route'lar aynı pool nesnesini require() ile alır	Uygulama boyunca tek PostgreSQL bağlantı havuzu; kaynak israfı ve bağlantı sızıntısı önlenir; bağlantı sayısı veritabanı limiti dahilinde kalır
Module (CommonJS)	Tüm backend dosyaları	Her dosya require/module.exports ile izole edilir; server.js tüm route'ları app.use() ile mount eder	Sorumluluklar dosya bazında ayrışır; bağımsız test ve bakım; değişiklik yayılımı minimize edilir
MVC (Model-	routes/ (Controller) + React pages/ (View) +	Route dosyaları HTTP katmanı, React sayfaları sunum katmanı,	Katmanların bağımsız geliştirilmesi ve test edilmesi; frontend değişikliği

View-Controller)	schema.sql (Model)	SQL şeması veri modelidir; katmanlar birbirinden bağımsız değiştirilebilir	backend'i, backend değişikliği veritabanını etkilemez
Facade	db/pool.js tüm route'lara ortak arayüz sunar	Tüm veritabanı erişimi pool.query() veya pool.connect() üzerinden yapılır; bağlantı kurulumu ve transaction detayları route'lardan gizlenir	Route modülleri pg.Pool konfigürasyonunu bilmek zorunda kalmaz; havuz değişikliği tek dosyada yapılır; test için mock pool kolayca enjekte edilebilir
Observer (Planlı — 2. iterasyon)	Socket.io sunucu + KDSEkrani.jsx istemcisi	OrdersController sipariş durumunu güncellerken socket.to('kitchen').emit('siparis-guncellendi', data) yayını yapar; KDSEkrani.jsx socket.on() ile dinler	Sipariş 'hazır' olduğunda yalnızca ilgili garson ve KDS ekranı bildirim alır; bileşenler yayıncı-abone modeli ile birbirinden bağımsız kalır; polling trafiği ortadan kalkar

Sözlük (Glossary)

Terim	Tanım
RYS	Restoran Yönetim Sistemi — bu projenin kısaltması
KDS	Kitchen Display System — Mutfak Ekran Sistemi; KDSEkrani.jsx bileşeniyle uygulanmıştır
POS	Point of Sale — Satış Noktası terminali; sipariş ve ödeme için kullanılan dokunmatik cihaz
SRS	Software Requirements Specification — Yazılım Gereksinimleri Belgesi; proje analiz raporu
JWT	JSON Web Token — oturum kimlik doğrulama belirteci; 2. iterasyonda eklenecek
REST API	Representational State Transfer — HTTP üzerinden kaynak yönetimi web servisi standardı
SPA	Single Page Application — tek sayfa uygulaması; React.js (CRA) ile geliştirilmiştir
pg / node-postgres	Node.js için PostgreSQL istemci kütüphanesi (v8.20); ham SQL sorguları ve bağlantı havuzu
pool.query(sql, params)	pg havuzundan geçici client alıp sorgu çalıştıran yöntem; basit işlemler için
pool.connect()	pg havuzundan transaction için özel client alma; BEGIN/COMMIT/ROLLBACK zincirinde kullanılır
client.release()	pg transaction client'ını havuza iade etme; finally bloğunda çağrılması zorunludur
ACID	Atomicity Consistency Isolation Durability — orders.js transaction yapısıyla PostgreSQL seviyesinde sağlanır
BEGIN / COMMIT / ROLLBACK	PostgreSQL transaction kontrolü komutları; sipariş oluşturmada veri bütünlüğünü garanti eder
ON DELETE CASCADE	FK kısıtlaması; üst kayıt silinince bağlı tüm alt kayıtlar otomatik silinir; orphan önlenir
BCrypt	Şifre hash algoritması; sifre_hash sütununa 2. iterasyonda uygulanacak; cost factor 10
hatali_giris	personel tablosunda başarısız giriş sayacı (INT); 5 olunca kilit_bitis set edilir
kilit_bitis	personel tablosunda hesap kilidi bitiş zamanı (TIMESTAMP); bu tarihten önce giriş reddedilir
stok_miktar	urunler tablosundaki anlık stok adedi; sipariş sonrası azaltılır (2. iterasyon)
kritik_seviye	urunler tablosunda e-posta uyarı eşiği; stok_miktar ≤ kritik_seviye ise mevcut=false
mevcut	urunler.mevcut BOOLEAN; false ise sipariş ekranında ürün 'Tükendi' olarak gösterilir
Singleton	Uygulama boyunca tek nesne örneği; db/pool.js bu deseni CommonJS module.exports ile uygular

Facade	Karmaşık alt sistemi tek arayüzle gizleme deseni; pool.js route'lara Facade sağlar
MVC	Model-View-Controller — sorumlulukları 3 katmana ayıran mimari desen
Observer	Yayıncı-abone deseni; Socket.io 2. iterasyonda bu desen ile KDS güncellemelerini sağlayacak
CRA	Create React App — React projesinin build toolchain'i; react-scripts 5.0.1 kullanılmaktadır
KDV	Katma Değer Vergisi; odemeler.tutar alanında KDV dahil toplam tutar olarak saklanmaktadır
E-Fatura	Türk Vergi Mevzuatı'na uygun elektronik fatura; fatura tablosu 2. iterasyonda eklenecek

Referanslar

1. Bruegge, B. & Dutoit, A. H. (2004). Object-Oriented Software Engineering: Using UML, Patterns, and Java (2nd ed.). Prentice-Hall. ISBN: 0-13-047110-0.

2. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. ISBN: 0-201-63361-2.

3. Tüzün, E. — CS 319 Object-Oriented Software Engineering Ders Sunumları. Bilkent Üniversitesi. https://www.cs.bilkent.edu.tr/~eraytuzun/teaching/cs319/

4. Söylev, A. — BİL 204 Yazılım Mühendisliği Ders Sunumları. Necmettin Erbakan Üniversitesi, 2026.

5. Örnek Tasarım Raporu — Fall 2022. https://www.cs.bilkent.edu.tr/~eraytuzun/teaching/cs319/SampleProjects/Fall2022-Sample-DesignReport.pdf

6. Örnek Tasarım Raporu — LOST (Fall 2021). https://github.com/yitepeli/CS319Project_Group2D_LOST/blob/master/Reports/Submissions/DesignReport/Design_Report_Final.pdf

7. RYS Kaynak Kodu. GitHub. https://github.com/rmznfaruk/rys-bil204

8. Express.js Dokümantasyonu (v5). https://expressjs.com/en/guide/

9. node-postgres (pg) Dokümantasyonu. https://node-postgres.com/

10. React.js Dokümantasyonu (v19). https://react.dev/

11. PostgreSQL 15 Dokümantasyonu. https://www.postgresql.org/docs/15/

12. Socket.io Dokümantasyonu (v4). https://socket.io/docs/v4/ (2. iterasyon için planlanmış)